DORMITORY MANAGEMENT PLATFORM

Complete Project Documentation

Author: Begimbaev Navruzbek

Year: 2026

Repository: github.com/NawrizbekBegimbaev/Domitory_BackEnd

Production: https://begimbaev-dormitory.uk

Chapter 1

INTRODUCTION

1.1 Introduction

1.1.1 Brief Overview

In modern universities, the number of dormitories and the students living in them grows every year. However, most institutions still keep records of residents, room assignments, payments, and visit logs manually — in paper notebooks and Excel spreadsheets. This approach leads to errors, lost data, duplication, and the inability to quickly retrieve up-to-date information about bed occupancy and outstanding debt.

The **Dormitory** platform automates the work of dormitory administration: managing residents, contracts, room assignments, billing, reporting, and audit — all in a single web and mobile application.

1.1.2 Dormitory Platform

"Dormitory" is a commercial SaaS platform for managing university dormitories. It replaces paper-based document workflow and provides the dorm manager, accountant, and administrator with a unified tool for all operations.

Key features:

- Management of buildings, floors, and rooms with pricing
- Tracking of residents, guardians, and documents
- Contract lifecycle: creation → assignment → transfer → termination
- Automatic billing based on a pro-rata rule
- FIFO-based payment allocation
- 6 types of reports (occupancy, debtors, available rooms, payments, residents, summary)
- Audit log for all financial and operational actions
- Telegram bot for OTP-based login
- Mobile application for administrators (iOS + Android)
- Multilingual interface: Russian, Uzbek, Karakalpak

1.1.3 Scope

The platform works on any modern device with a browser (Chrome, Safari, Firefox, Edge). The mobile app supports iOS 13+ and Android 8+. The server-side is deployed on Hetzner CPX22 (3 vCPU, 4GB RAM, 80GB SSD), running Ubuntu 24.04. Access is provided via the domain **begimbaev-dormitory.uk** with SSL from Cloudflare.

Benefits:

- **Improved Efficiency.** The dorm manager no longer wastes time searching journals — all data is available in 2 clicks.

- **Reduced Costs.** Automatic billing and payment allocation save the accountant hours of work daily.
- **Enhanced Collaboration.** Administrator, accountant, and dorm manager see one source of truth simultaneously.
- **Transparency for Residents.** Any student can check their balance and payment history at any time.
- **Full Audit.** Every significant action is recorded in an immutable log.

1.2 Introducing Dormitory System

1.2.1 Problem Statement

The current dormitory management system in most universities of Uzbekistan is characterized by inaccuracy, unreliability, and inefficient use of time, money, and human resources. Assignment journals get lost, billing is calculated manually with errors, there is no single source of truth about actual occupancy, and it is impossible to quickly produce a debtors report.

1.2.2 Current System

In most dormitories, records are kept by a combination of:

- Paper notebooks held by the dorm manager
- Excel spreadsheets with the resident registry
- A handwritten cash journal
- Payment receipts kept by residents themselves
- Verbal agreements about transfers between rooms

This approach requires significant amounts of paper, archive cabinets, manual processing time, and constant data integrity checks.

1.2.3 Drawbacks of the Current System

1. **Inaccuracy.** The human brain cannot reliably hold the state of hundreds of residents — discrepancies between the journal and reality are inevitable.
2. **Data loss.** Notebooks tear, get lost, or get water-damaged; Excel files are overwritten and have no history.
3. **Time.** Searching for one resident in the archive takes 10–30 minutes.
4. **No analytics.** It is impossible to answer "how many beds are free on the 3rd floor of building #2" within a minute.
5. **No audit trail.** It is impossible to determine who made a change or accepted a payment, and when.

6. **Billing errors.** When transferring residents, charges are recalculated manually and often contain mistakes.

1.2.4 Working of System Proposal

The main goal is to develop an intelligent platform that:

- Stores personal data of residents, guardians, and documents
- Manages the lifecycle of contracts and assignments
- Automatically generates monthly charges
- Allocates payments via FIFO
- Allows transferring residents in one click with automatic recalculation
- Refunds overpayments upon contract termination
- Provides 6 built-in reports
- Logs all critical operations to an audit log
- Is accessible from a computer and a smartphone
- Supports three interface languages
- Has a role-based permission model

1.2.5 Tools & Technologies

Component	Technology
Backend	Python 3.12+, Django 4.2 LTS, Django REST Framework 3.15+
Database	PostgreSQL 16+
Authentication	SimpleJWT 5.3+
API documentation	drf-spectacular (OpenAPI 3.0)
Frontend	React 19, Vite, TypeScript, Tailwind CSS v4
Mobile application	Flutter 3.41+, Dart 3.11+, Provider
Telegram	requests + Bot API
Tests	pytest, pytest-django, factory-boy, Playwright
WSGI / Web server	Gunicorn + Nginx
Deployment	Hetzner Cloud, Cloudflare DNS/SSL
IDE	PyCharm, VS Code, Xcode, Android Studio

1.3 Feasibility Study

A feasibility study is an estimate of whether the identified user needs can be satisfied using current software and hardware technologies, and whether the project will be effective

from a business standpoint within the given budgetary constraints.

1.3.1 Economic Feasibility

Development costs are minimal: an open-source stack (Django, React, Flutter, PostgreSQL — all free) is used. Server infrastructure costs €15.5/month for Hetzner CPX22, plus $10/year for the domain. ROI: deploying in a single dormitory (~400 beds) saves the university one full-time clerk position.

1.3.2 Technical Feasibility

Development environment	**macOS, Linux, Windows**
Test OS	Ubuntu 24.04, iOS 13+, Android 8+
Implementation tools	PyCharm, VS Code, Xcode
Testing	pytest, Playwright
Documentation	Markdown, drf-spectacular

All technologies are stable, have large communities, and provide long-term support.

1.3.3 Operational Feasibility

The system covers all operational scenarios of a dormitory: from move-in to move-out, from charge generation to refund of overpayments. Performance — API response time <200ms on a typical request with 400 residents.

1.3.4 Legal and Ethical Feasibility

The system does not infringe on intellectual property rights (only open-source software is used). Personal data is stored on a server in Finland (Hetzner) with encryption at rest and in transit. Access is restricted by roles.

1.3.5 Schedule Feasibility

Phase	**Duration**	**Status**
1. Web platform (backend + frontend)	4 months	DONE
1.5. Mobile app for administrators	1.5 months	DONE
2. Mobile app for students	2 months	Not started
3. Online payments (Payme/Click)	1 month	Not started

Chapter 2

SYSTEM ANALYSIS

2.1 System Analysis

Analysis is the main part of project development. Most of the project time is spent on analysis. The Dormitory project used a combination of prototyping and an object-oriented methodology, applying RUP (Rational Unified Process).

2.2 Analysis

The analysis is divided into two parts:

- Requirement Analysis
- Domain Analysis

2.2.1 Requirement Analysis

The purpose is to build a model of the system's behavior. An object-oriented approach is used: requirements are examined from the perspective of the classes and objects of the dormitory domain.

Functional Requirements

Authentication and Users

- Login with email + password
- Login with phone number + OTP via Telegram
- Password reset via email
- 5 roles: platform_admin, university_admin, dorm_manager, accountant, security_staff
- CRUD of users with different roles

Inventory

- Management of buildings, floors, and rooms
- Setting monthly price per room
- Gender attribute for room residents
- Listing available rooms with filtering

Residents

- CRUD of residents with full name, passport, contacts, faculty
- Linking guardians and documents
- Viewing the current balance of a resident
- Transferring a resident to another room with recalculation
- Terminating a contract with refund of overpayment

Contracts and Assignments

- Creating residency contracts
- Assigning a room with auto-generation of monthly charges
- Closing an assignment

Billing

- Automatic charge generation based on the move-in date
- Pro-rata charge for partial months
- Full-room purchase (single occupancy)
- Payment registration with automatic FIFO allocation
- Refund of overpayments upon move-out

Reports

- Occupancy
- Available rooms
- Debtors
- Payments history
- Resident registry
- Summary report

Audit

- All financial transactions, residency changes, and contract changes are logged immutably

Non-Functional Requirements

- **Portability.** The web interface works in any modern browser. The mobile app runs on iOS and Android.
- **Performance.** API response <200ms at the 95th percentile under loads up to 1000 RPS.
- **Security.** JWT tokens with short TTL, refresh tokens, role-based permissions, HTTPS-only, CSRF/XSS protection.
- **Localization.** Three languages: Russian, Uzbek, Karakalpak.
- **Documentation.** OpenAPI 3.0 + Markdown.

2.2.2 Domain Analysis

Rational Unified Process (RUP) is used — an iterative process of 4 phases:

Inception — defined business rationale (replacing paper-based records in Uzbek university dormitories) and project scope.

Elaboration — gathered use cases, conducted interviews with the future user (dormitory manager), identified risks (unacceptable loss of financial data), and built the baseline architecture (Django REST + PostgreSQL + React + Flutter).

Construction — implementation as a series of mini-projects: each Django app (accounts, inventory, residents, occupancy, billing, reports, audit) went through its own analysis → design → coding → testing → integration cycle.

Transition — beta testing in production (begimbaev-dormitory.uk), performance tuning, and writing the user manual.

Actors

- **Platform Admin** — global platform administrator
- **University Admin** — university-level administrator
- **Dorm Manager (Comendant)** — dormitory manager
- **Accountant** — accountant
- **Security Staff** — security guard (read-only)

2.3 Use Cases

A use case is a specific way of using the system through part of its functionality. Each use case is initiated by an actor and describes the interaction between the actor and the system.

2.3.1 Use Case Analysis

The Dormitory project identifies more than 30 use cases. The key ones are described below.

2.4 Use Case Diagram

2.4.1 Login (Email + Password)

Use Case	**Login**
Use Case #	2.4.1
Actors	All roles
Purpose	Authentication in the system
Pre-condition	User is registered
Flow	1. Enter email and password → 2. POST /auth/login/ → 3. Receive JWT access + refresh → 4. Land on dashboard
Post-condition	Session is active, tokens are stored

2.4.2 Login via Telegram OTP

Use Case	Phone Login
Use Case #	2.4.2
Actors	All roles
Purpose	Passwordless login via Telegram
Flow	1. Enter phone number → 2. OTP is sent to Telegram bot → 3. Enter 6-digit code → 4. Receive JWT

2.4.3 Add Resident

Use Case	Add Resident
Use Case #	2.4.3
Actors	Dorm Manager, University Admin
Purpose	Adding a new resident
Flow	1. Open form → 2. Fill in name, passport, faculty, contacts → 3. POST /residents/ → 4. Add guardian → 5. Attach documents

2.4.4 Assign Room (Move-in)

Use Case	Assign Room
Actors	Dorm Manager
Purpose	Assigning a resident to a room
Pre-condition	Resident and room exist, free bed is available, contract is active
Flow	1. Select resident → 2. Select room with a free bed → 3. Set move-in date → 4. Create RoomAssignment → 5. Auto-generate Charge records → 6. Create StayRecord → 7. AuditLog

2.4.5 Transfer Resident

Use Case	Transfer Resident
Actors	Dorm Manager
Flow	1. Close current RoomAssignment → 2. Open new one in another room → 3. Recalculate charges (old room — pro-rata, new — from transfer date) → 4. StayRecord → 5. AuditLog

2.4.6 Register Payment (FIFO)

Use Case	Register Payment
Actors	Accountant
Flow	1. Select resident → 2. Enter amount and date → 3. POST /payments/ → 4. Auto-allocate PaymentAllocation from oldest Charge to newest → 5. Update Charge statuses → 6. AuditLog

2.4.7 Terminate Contract (Move-out)

Use Case	Terminate Contract
Actors	Dorm Manager
Flow	1. Set termination date → 2. Close Contract → 3. Close Assignment → 4. Calculate overpayment → 5. Create refund Charge → 6. StayRecord → 7. AuditLog

2.4.8 Generate Report

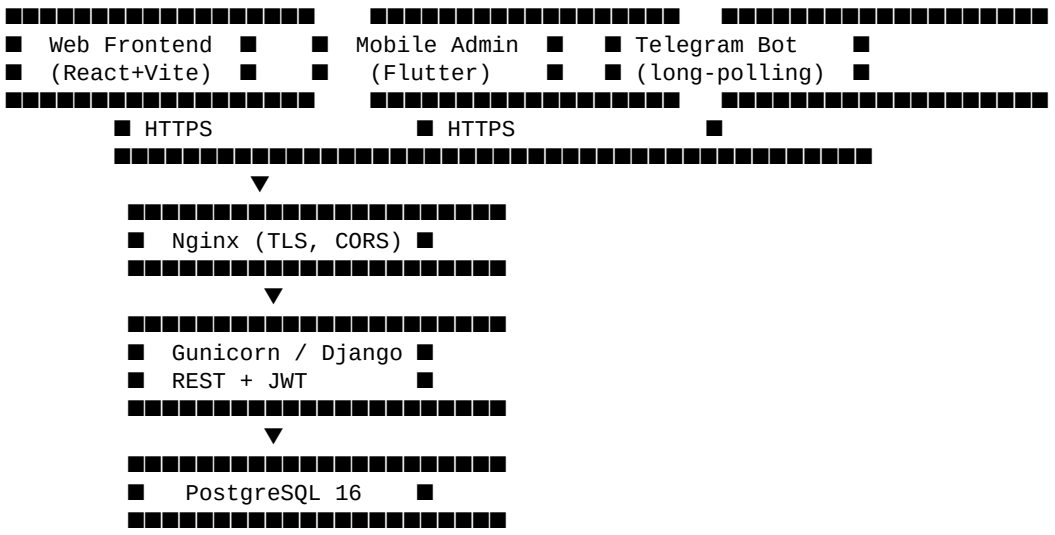
Use Case	Generate Report
Actors	All roles (with different permissions)
Flow	1. Choose report type → 2. Set period / filters → 3. GET /reports/{type}/ → 4. View or export

Chapter 3

SYSTEM DESIGN

3.1 System Design

The Dormitory architecture follows the **Fat Services, Thin Views** principle: all business logic is encapsulated in `services.py` of each Django app, while views only receive the request and call the service.



Backend Layers

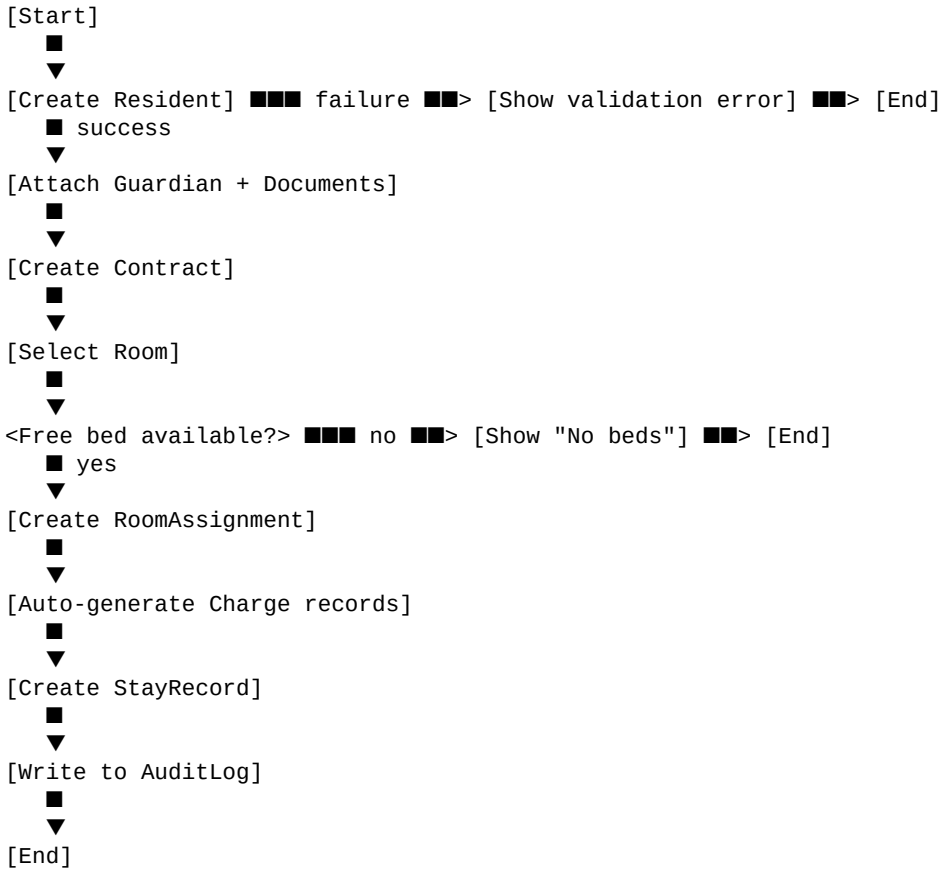
- **API layer** (views.py) — request handling, serialization, permissions
- **Service layer** (services.py) — business logic, transactions, invariant validation
- **Model layer** (models.py) — ORM models, migrations
- **Audit layer** (audit/services.py) — immutable logging

Django Apps

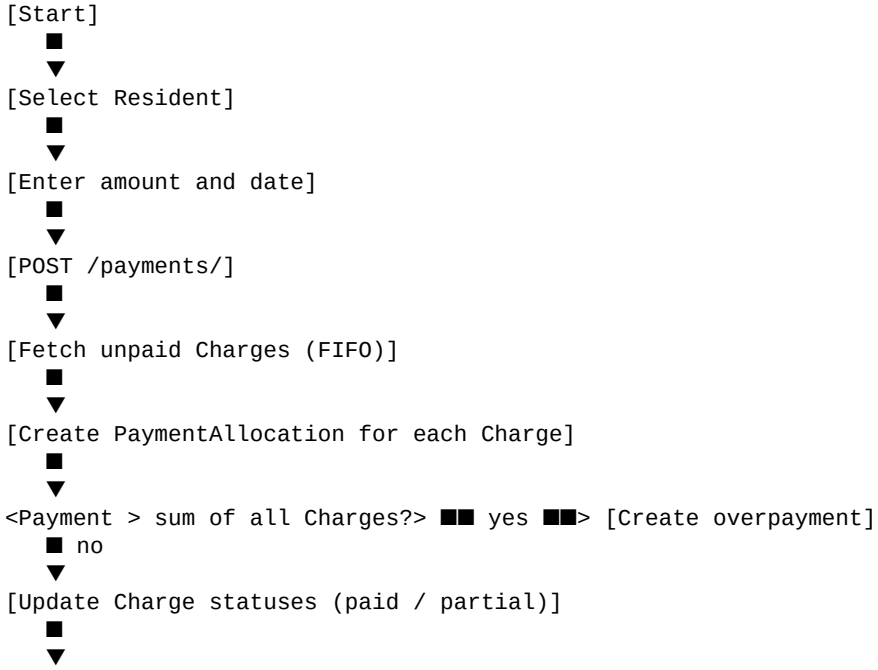
App	Purpose
`accounts`	Users, JWT, OTP, password reset
`inventory`	Buildings, floors, rooms
`residents`	Residents, guardians, documents, faculties
`occupancy`	Contracts, assignments, transfers, history
`billing`	Charges, payments, FIFO allocation
`reports`	6 reports
`audit`	Audit log
`access_control`	Fine-grained permissions

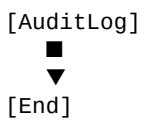
3.2 Activity Diagrams

Activity Diagram: Move-in



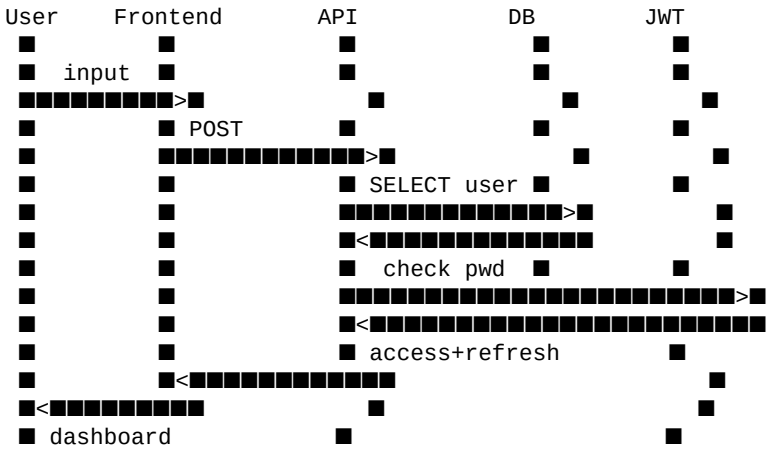
Activity Diagram: Payment Registration



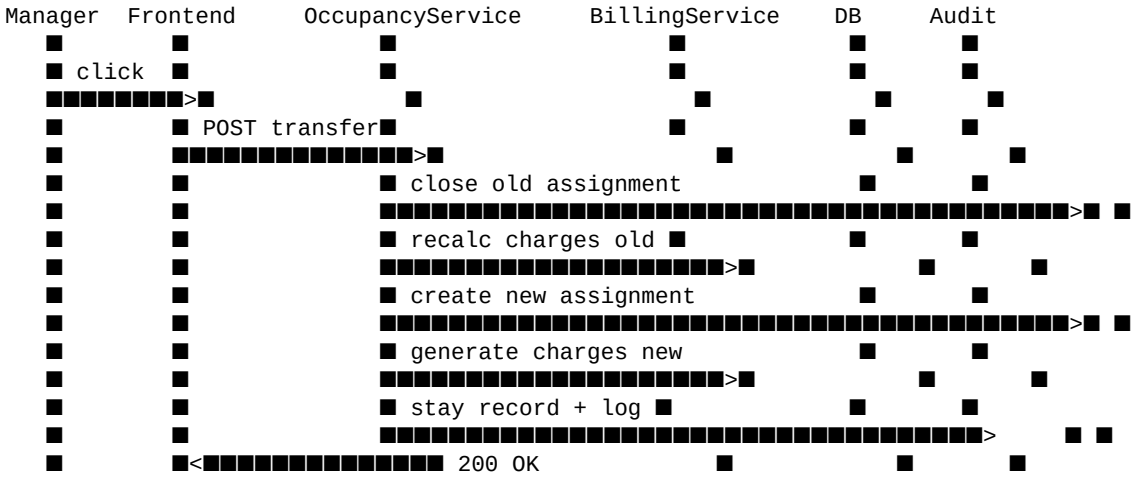


3.3 Sequence Diagram

Sequence: Login



Sequence: Resident Transfer



3.4 Detailed Sequence and Activity Diagrams

3.4.1 Authentication (JWT)

- Access TTL: 60 minutes
- Refresh TTL: 7 days
- Refresh rotation enabled

3.4.2 List Residents

- GET /residents/?page=1&search=&faculty=
- Cursor pagination at 25 items per page

3.4.3 Save Record (any resource)

- Validation → serializer → service → model → audit

3.4.4 Delete Record

- Soft-delete for financial entities
- Hard-delete only for non-critical reference data

Chapter 4

DATABASE DESIGN

4.1 Database System

The project uses **PostgreSQL 16** — a relational DBMS supporting ACID transactions, B-tree/GIN indexes, JSONB fields, and complex constraints. The choice is driven by maturity, reliability, and built-in Django support.

4.2 Database System (Description)

The database is hosted on the same Hetzner server as the backend. Production database size is ~120 MB for one dormitory. Backups run daily via `pg_dump`.

4.3 Advantages of Database

- ACID guarantees for financial transactions
- Cascading foreign keys (with explicit `on_delete=PROTECT` for critical links)
- Complex queries for reports via the ORM
- Full-text search (resident name)
- Transaction isolation for safe FIFO allocation

4.4 Normalization

All main tables are normalized to **3rd Normal Form**:

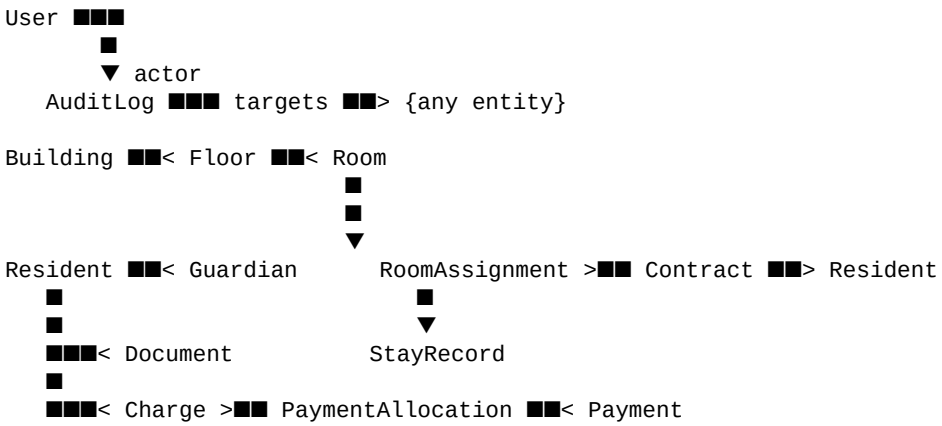
- 1NF — atomic values, no repeating groups
- 2NF — no partial dependencies on a composite key
- 3NF — no transitive dependencies

4.5 Database Design

Main Entities

Table	Key Fields
accounts_user	id, email (unique), phone (unique, null), telegram_id (unique, null), role, password_hash
inventory_building	id, name, address
inventory_floor	id, building_id, number
inventory_room	id, floor_id, number, capacity, monthly_price, gender
residents_resident	id, full_name, passport, faculty_id, phone, birth_date, gender
residents_guardian	id, resident_id, full_name, relation, phone
residents_document	id, resident_id, type, file, uploaded_at
residents_faculty	id, name
occupancy_contract	id, resident_id, start_date, end_date, status
occupancy_roomassignment	id, contract_id, room_id, start_date, end_date
occupancy_stayrecord	id, resident_id, action, room_id, timestamp
billing_charge	id, resident_id, period_start, period_end, amount, paid_amount, status
billing_payment	id, resident_id, amount, paid_at, method
billing_paymentallocation	id, payment_id, charge_id, amount
audit_auditlog	id, actor_id, action, object_type, object_id, payload, timestamp

4.5.1 Design View (simplified ER diagram)



4.6 Data Modeling

Key Invariants

1. **Contact uniqueness.** `User.email`, `User.phone`, `User.telegram_id` — unique (where not null).

2. **Capacity guard.** Sum of active RoomAssignments per room ≤ Room.capacity.
3. **FIFO allocation.** PaymentAllocations are created only against unpaid Charges, ordered by `period_start` ASC.
4. **Pro-rata charge.** If a resident moves in/out mid-month, Charge.amount is proportional to the days of stay.
5. **Immutable audit.** AuditLog records cannot be edited or deleted.
6. **Single-tenant.** The Organization entity has been removed — all data is in a single namespace.

Indexes

- `accounts_user(email)` — UNIQUE
- `accounts_user(phone)` — UNIQUE WHERE NOT NULL
- `residents_resident(passport)` — UNIQUE
- `billing_charge(resident_id, period_start)` — composite
- `audit_auditlog(timestamp DESC)` — for fast retrieval of latest entries

Chapter 5

IMPLEMENTATION

5.1 Languages

5.1.1 Python (Backend)

Python 3.12+ — a high-level language with a rich web-development ecosystem. Used for the entire backend through Django.

Advantages:

- Mature web framework (Django) with built-in ORM, migrations, and admin
- DRF for RESTful API out of the box
- Huge community and stable LTS releases
- Easy testing (pytest + factory-boy)

5.1.2 TypeScript (Frontend)

React 19 + TypeScript + Vite — a modern frontend stack.

- Strict typing protects from runtime errors
- Vite provides instant HMR during development
- Tailwind CSS v4 — utility-first styling without giant CSS files
- React 19 — server components and optimized rendering

5.1.3 Dart (Mobile)

Flutter 3.41+ / Dart 3.11+ — a cross-platform mobile stack. Single codebase for both iOS and Android.

Reasons for Flutter:

- High performance (compiles to native code)
- Provider — simple and predictable state management
- Rich Material Design widget set
- Hot Reload accelerates development

5.1.4 Reason for Stack Choice

Unlike a pure Java or Swift approach, **Django + React + Flutter** was chosen because:

- One backend serves both web and mobile
- Business logic is not duplicated
- Model migrations don't require synchronization between two codebases
- Open-source, no licensing fees

5.2 Software Selection

Type	Tool
Backend IDE	PyCharm / VS Code
Frontend IDE	VS Code
Mobile IDE	Xcode (iOS), Android Studio (Android), VS Code (Dart)
Version control	Git + GitHub
Browser testing	Playwright
API tests	pytest + requests
Documentation	Markdown + drf-spectacular
Deployment	rsync + systemd + Nginx
Monitoring	systemctl + journalctl

5.3 Module Implementation

Backend Modules

```
domitory/apps/
■■■ accounts/         # Authentication (JWT, OTP, password reset)
■■■ inventory/        # Buildings, floors, rooms
■■■ residents/        # Residents, guardians, documents
■■■ occupancy/        # Contracts and assignments (move-in/transfer/move-out)
■■■ billing/          # Charges and payments (FIFO)
■■■ reports/          # 6 reports
■■■ audit/            # Audit log
■■■ access_control/   # Fine-grained permissions
```

Frontend Pages (14)

LoginPage, DashboardPage, BuildingsPage, RoomsPage, ResidentsPage, ResidentDetailPage, ContractsPage, AssignmentsPage, BillingPage, PaymentsPage, ReportsPage, UsersPage, AuditPage, AccessPage

Mobile Screens (10)

LoginScreen, DashboardScreen, ResidentsScreen, ResidentDetailScreen, RoomsScreen, FullRoomScreen, ReportsScreen, UsersScreen, MenuScreen, HomeShell

Chapter 6

SYSTEM TESTING

6.1 Verification and Validation

6.1.1 Verification

Verification — "are we building the product right?" Checks that the implementation matches the specification.

In Dormitory, verification is ensured by:

- Type checking (TypeScript on the frontend, type hints in Python)
- 249 backend unit tests via pytest
- Code review for every change before merging

6.1.2 Validation

Validation — "are we building the right product?" Checks that the product matches user requirements.

Ensured by:

- 72 API tests (qa/test_api.py)
- 50 E2E tests via Playwright (qa/test_e2e.py)
- Beta testing in a real dormitory

6.2 Testing

6.2.1 Debugging

- Django Debug Toolbar in dev environment
- Sentry-style error logging in production
- pdb / ipdb for step-by-step debugging

6.2.2 Overview of Testing

Tests are split into 3 levels:

1. **Unit** — individual functions and services (249 tests)
2. **API** — REST endpoints (72 tests, locally and in production)
3. **E2E** — user scenarios in a browser (50 tests, Playwright)

6.2.3 Objective of Testing

- Guarantee that billing logic is correct (FIFO, pro-rata)
- Verify that access permissions are respected
- Ensure migrations apply without data loss

- Protect against regressions during refactoring

6.3 Testing Strategies

6.3.1 White Box Testing

Testing with full knowledge of the internal code structure.

6.3.1.1 Class Level Testing

Each service (`apps/billing/services.py`, `apps/occupancy/services.py`) is covered by unit tests.

```
def test_fifo_allocation():
    resident = ResidentFactory()
    ChargeFactory(resident=resident, amount=1000, period_start='2026-01-01')
    ChargeFactory(resident=resident, amount=1000, period_start='2026-02-01')
    payment = PaymentService.register(resident, amount=1500, paid_at='2026-02-15')
    assert payment.allocations.first().amount == 1000  # old Charge
    assert payment.allocations.last().amount == 500    # new Charge
```

6.3.1.2 Component Level Testing

API tests via pytest + DRF APIClient.

```
def test_create_assignment_generates_charges(api_client):
    resp = api_client.post('/api/v1/assignments/', {...})
    assert resp.status_code == 201
    assert Charge.objects.filter(resident=resident).count() == 12
```

6.3.1.3 Assembly Level Testing

Integration tests covering several services together: Contract → Assignment → Charge → Payment → AuditLog.

6.3.2 Black Box Testing

E2E via Playwright — without internal knowledge, simulating user actions.

```
def test_e2e_resident_creation_flow(page):
    page.goto(f"{BASE_URL}/login")
    page.fill('input[name=email]', 'admin@dorm.uk')
    page.fill('input[name=password]', '...')
    page.click('button:has-text("Login")')
    page.click('text=Residents')
    page.click('text=Add')
    # ... fill form ...
    expect(page.locator('text=Resident created')).to_be_visible()
```

6.4 Test Cases

#	Scenario	Expected Result	Status
TC-001	Login with correct password	200 + JWT tokens	PASS
TC-002	Login with wrong password	401	PASS
TC-003	OTP login via Telegram	OTP received and confirmed	PASS
TC-004	Create resident with unique passport	201	PASS
TC-005	Create resident with duplicate passport	400	PASS
TC-006	Assign room with free bed	201 + auto-charges	PASS
TC-007	Assign room when full	400 "No beds"	PASS
TC-008	Transfer resident	Old charges recalculated, new ones created	PASS
TC-009	Payment allocates by FIFO	Old Charges paid first	PASS
TC-010	Terminate contract with overpayment	Refund Charge created	PASS
TC-011	security_staff editing access	403	PASS
TC-012	Audit log records all financial actions	Entry present	PASS
TC-013	"Debtors" report returns correct list	List matches manual calculation	PASS
TC-014	Password reset via email	Email sent, token valid	PASS
TC-015	Mobile app syncs with prod API	Data matches web interface	PASS

Testing Results

Level	Count	Pass	Fail
Unit (pytest)	249	249	0
API (qa/test_api.py)	72	72	0
E2E (Playwright)	50	50	0
Total	**371**	**371**	**0**

Chapter 7

FUTURE ENHANCEMENT

7.1 Planned Enhancements

7.1.1 Student Mobile Application (Phase 2)

A separate app for residents allowing them to:

- View balance and payment history
- Submit maintenance requests
- Book the laundry machine
- Receive notifications about dormitory events

7.1.2 Online Payments (Phase 3)

Integration with **Payme** and **Click** to accept payments directly from residents:

- Webhook on every successful payment
- Automatic allocation through the existing FIFO logic
- Notification to the accountant via Telegram

7.1.3 Push Notifications

Through Firebase Cloud Messaging:

- Reminders about upcoming payments
- Notifications about new charges
- Messages from the administration

7.1.4 Mail Inbox / Outbox

Internal mail between administration and residents, with conversation history.

7.1.5 Automatic Passport Recognition

OCR upon uploading a passport scan, with auto-fill of fields.

7.1.6 Multi-tenant SaaS

Bringing back the Organization entity to serve multiple universities from a single instance. Data isolation via row-level security in PostgreSQL.

7.1.7 Analytical Dashboard

- Occupancy forecasting
- Debt analysis by faculty
- Dormitory map with visualization of free beds

7.1.8 Telegram Bot for Residents

Commands `/balance`, `/payments`, `/contract` — for fast access without an app.

Chapter 8

USER MANUAL

8.1 Accessing the System

Web Application

Open **https://begimbaev-dormitory.uk** in your browser and log in with the credentials provided.

Mobile Application

iOS — via TestFlight, Android — APK. After installation, choose "Production" in API URL settings.

8.2 First Login

1. Enter your email and the temporary password (sent by the administrator)
2. The system will prompt you to change your password
3. After changing the password, you'll land on the Dashboard

8.3 Dashboard

The home page shows:

- Total number of residents
- Occupancy in %
- Total debt amount
- Latest operations

8.4 Adding a New Resident

1. **Residents** → **Add.** Fill in name, passport, faculty, contacts.
2. **Guardians** → **Add Guardian** for the created resident.
3. **Documents** → **Upload** scan of passport and application.
4. **Contracts** → **Create Contract** with start and end dates.
5. **Move-in** → **Select Room** with a free bed.
6. The system automatically creates monthly charges.

8.5 Registering a Payment

1. **Finance** → **Payments** → **New Payment.**
2. Select the resident.
3. Enter amount and date.

4. Save — the system automatically allocates the amount via FIFO.

8.6 Transferring a Resident

1. Open the resident's card → **Transfer.**
2. Select the new room and date.
3. Confirm — old charges are recalculated, new ones are created.

8.7 Terminating a Contract

1. Open the contract → **Terminate.**
2. Set the termination date.
3. If there is overpayment — a refund charge is created.
4. AuditLog records the operation.

8.8 Reports

Reports in the side menu → choose type:

- **Occupancy** — % occupancy by buildings
- **Available rooms** — free beds with filters
- **Debtors** — list of residents with debt
- **Payments** — payment registry for a period
- **Residents** — registry of all residents
- **Summary** — overall statistics

8.9 Mobile Application

All web features are available in the mobile app: login (email/password or phone/OTP), Dashboard, Residents (CRUD + details), Rooms (view), Reports, Audit, Users, Menu/Profile.

8.10 Troubleshooting

Issue	Solution
OTP not received	Check that the Telegram bot @BegimbaevDormitoryBot is not blocked
403 on action	Your role doesn't have permission. Contact the administrator
Balance discrepancy	Check the AuditLog — there may have been a manual refund
App can't connect	Check internet connection and API URL

Conclusion

The **Dormitory** platform is a working, production-deployed commercial system. Test coverage — 371 tests (100% pass). Deployed on Hetzner Cloud, domain hosted on Cloudflare with SSL. Daily database backups. Roadmap — adding the student mobile app and online payments.

End of document